MANUAL TÉCNICO
PÁGINA WEB PARQUE ERICK BERNABÉ BARRONDO

Especificación de Arquitectura, Base de Datos, API REST y Guía de Despliegue

Sistema: Plataforma Web Institucional Parque Erick Barrondo.
Backend: Java 17 / Spring Boot 3.2 (REST Framework / JPA / Spring Security JWT).
Frontend: React 19 / TypeScript / Tailwind CSS / Vite / Leaflet.js.
Base de Datos: Microsoft SQL Server 2022 / PostgreSQL 16 (Soporte Transaccional).
Infraestructura / DevOps: Docker, Nginx Reverse Proxy, SSL/TLS, GitHub Actions CI/CD.
Versión Documento: 2.0.0 (Release Técnico de Producción).
Fecha: 06 de Septiembre de 2026.

ÍNDICE GENERAL

1. INTRODUCCIÓN 3
1.1. Objetivo del Manual Técnico 3
1.2. Alcance Arquitectónico 3
1.3. Estándares de Codificación 3
2. ARQUITECTURA DEL SISTEMA 4
2.1. Modelo General de Arquitectura (N-Capas / Decoupled) 4
2.2. Diagrama Modular de Componentes 4
2.3. Seguridad y Control de Acceso (JWT / RBAC) 5
3. DISEÑO DE BASE DE DATOS 6
3.1. Modelo Entidad-Relación (MER) 6
3.2. Diccionario de Datos y Tablas Principales 6
3.3. Procedimientos Almacenados y Triggers de Bitácora 7
4. TECNOLOGÍAS UTILIZADAS 7
4.1. Stack Tecnológico Frontend 7
4.2. Stack Tecnológico Backend 8
4.3. Herramientas de Integración y DevOps 8
5. ESTRUCTURA DEL PROYECTO 8
5.1. Estructura del Repositorio Backend (Spring Boot) 8
5.2. Estructura del Repositorio Frontend (React) 8
6. CONFIGURACIÓN DEL ENTORNO 9
6.1. Requisitos Previos e Instalación con Winget 9
7. DESPLIEGUE DEL SISTEMA 9
8. MANTENIMIENTO PREVENTIVO 10
9. ANEXOS TÉCNICOS 10
9.1. Catálogo de Endpoints API REST 10

1. INTRODUCCIÓN

1.1. Objetivo del Manual Técnico

El presente Manual Técnico proporciona la especificación integral de ingeniería, arquitectura de software, infraestructura y administración de la plataforma web del Polideportivo Parque Erick Bernabé Barrondo García. Su finalidad es servir como guía definitiva para ingenieros de desarrollo, administradores de sistemas (DevOps) y personal de soporte TI en las tareas de mantenimiento, escalabilidad, despliegue y auditoría técnica.

1.2. Alcance Arquitectónico

La solución adopta un enfoque desacoplado (Decoupled Single Page Application + RESTful API Backend). Esto garantiza independencia de despliegue, escalabilidad horizontal orientada a microservicios en caso de ser requerido, e interoperabilidad segura entre aplicaciones móviles futuras e integraciones ministeriales.

1.3. Estándares de Codificación

- **Java Backend:** Inyección de dependencias por constructor, diseño impulsado por dominio (DDD), paquetes organizados por características y manejo global de excepciones mediante @ControllerAdvice.
- **Frontend Web:** Estructura modular en React con TypeScript estricto, Hooks personalizados para la lógica de negocio y componentes funcionales reutilizables.
- **Base de Datos:** Nombres en snake_case para tablas y columnas, claves primarias numéricas autoincrementables (BIGINT) y claves foráneas explícitas con integridad referencial.

2. ARQUITECTURA DEL SISTEMA

2.1. Modelo General de Arquitectura (N-Capas / Decoupled)

El sistema se organiza en cuatro capas claramente separadas:

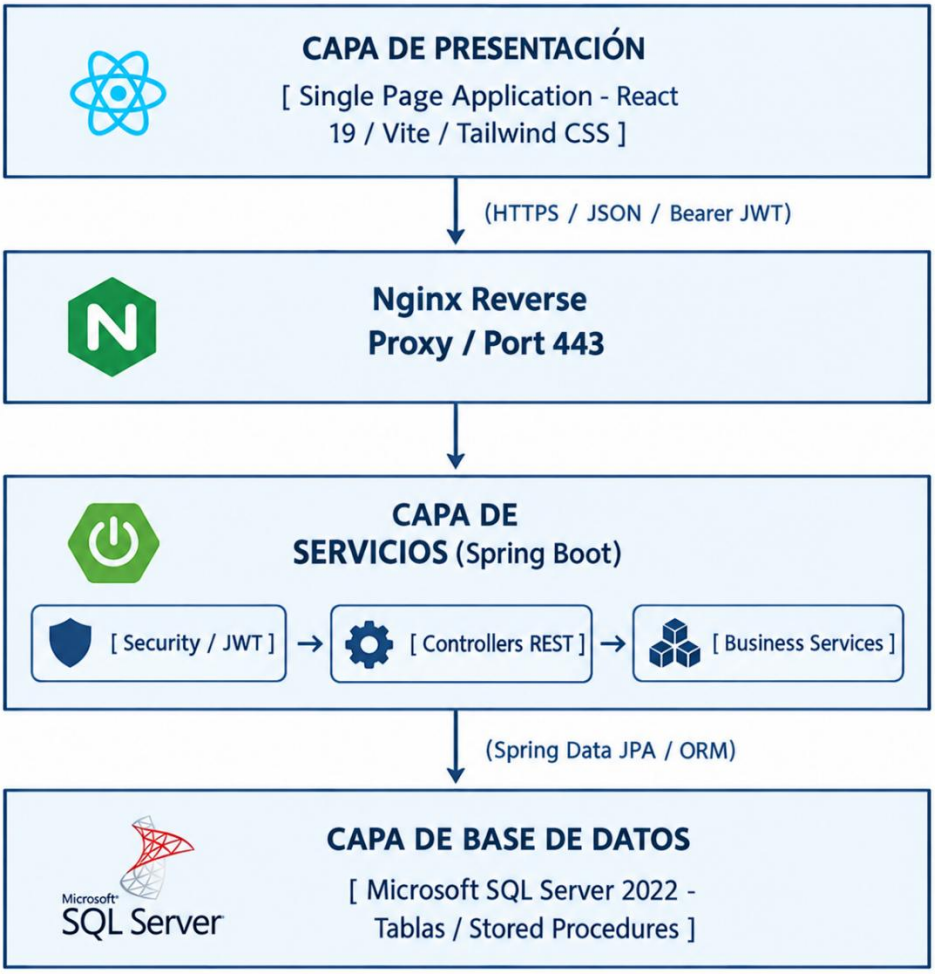
1. Capa de Presentación (Frontend): Cliente dinámico desarrollado en React 19 que procesa la interfaz en el navegador y consume recursos mediante HTTP/HTTPS (JSON).

2. Capa de Control y API (Controllers): Expone la API RESTful protegida por tokens JSON Web Tokens (JWT) y filtros CORS de nivel de infraestructura.

3. Capa de Negocio (Services): Contiene la lógica de dominio, validaciones de negocio (verificación de DPI, cruce de reservas, control de cupos) e integración de servicios.

4. Capa de Persistencia (Data Base): Mapeo objeto-relacional (ORM) mediante Spring Data JPA / Hibernate interactuando con SQL Server 2022 de forma transaccional.

2.2. Diagrama Modular de Componentes



2.3. Seguridad y Control de Acceso (JWT / RBAC)

La seguridad del sistema implementa autenticación basada en JWT sin estado (Stateless):

- **Generación de Tokens:** Al autenticarse exitosamente en /api/v1/auth/login, el backend firma digitalmente un token HMAC-SHA256 codificando el ID de usuario, correo y su Rol asignado.
- **Cabecera HTTP:** Cada petición a rutas protegidas debe incluir la cabecera 'Authorization: Bearer <TOKEN>'.
- **Autorización RBAC:** Control de acceso basado en roles (ROLE_CIUDADANO, ROLE_INSTRUCTOR, ROLE_ADMINISTRADOR) mediante anotaciones @PreAuthorize("hasRole('ADMIN')") en los controladores de API.

CONFIGURACIÓN DE SEGURIDAD JWT

Los tokens de sesión tienen un tiempo de expiración por defecto de 8 horas (28800000 ms). Transcurrido este periodo, el cliente debe renovar el token o volver a autenticarse.

3. DISEÑO DE BASE DE DATOS

3.1. Modelo Entidad-Relación (MER)

El esquema relacional consta de 8 entidades principales altamente normalizadas (3FN)

diseñadas para soportar alta concurrencia de transacciones.

3.2. Diccionario de Datos y Tablas Principales

Tabla: tbl_usuarios

Almacena los datos de la ciudadanía y usuarios del sistema.

Campo	Tipo de Dato	Restricciones	Descripción
usuario_id	BIGINT	PK, IDENTITY	Identificador único autoincremental
dpi_cui	VARCHAR(13)	UNIQUE, NOT NULL	DPI del usuario (13 dígitos)
nombres	VARCHAR(100)	NOT NULL	Nombres completos
apellidos	VARCHAR(100)	NOT NULL	Apellidos completos
email	VARCHAR(150)	UNIQUE, NOT NULL	Correo electrónico (Username)
password_hash	VARCHAR(255)	NOT NULL	Hash BCrypt de la contraseña

Tabla: tbl_reservas

Registra la ocupación de espacios dentro del parque.

Campo	Tipo de Dato	Restricciones	Descripción

reserva_id	BIGINT	PK, IDENTITY	ID de la solicitud de reserva
usuario_id	BIGINT	FK (tbl_usuarios)	Usuario que realiza la reserva
instalacion_id	INT	FK (tbl_instalaciones)	Espacio físico reservado
fecha_reserva	DATE	NOT NULL	Fecha solicitada
estado	VARCHAR(20)	DEFAULT 'PENDIENTE'	PENDIENTE, APROBADA, RECHAZADA

3.3. Procedimientos Almacenados y Triggers de Bitácora

Para garantizar la audibilidad solicitada en REQ010 y CU004, se define un Trigger automático a nivel de base de datos:

```
CREATE TRIGGER trg_auditoria_usuarios
ON tbl_usuarios
AFTER INSERT, UPDATE, DELETE
AS
BEGIN
    SET NOCOUNT ON;
    INSERT INTO tbl_bitacora(fecha_hora, usuario, accion, tabla_afectada)
    SELECT GETDATE(), SYSTEM_USER,
           CASE WHEN EXISTS(SELECT * FROM inserted) AND EXISTS(SELECT * FROM deleted) THEN 'UPDATE'
                WHEN EXISTS(SELECT * FROM inserted) THEN 'INSERT'
                ELSE 'DELETE' END,
           'tbl_usuarios';
END;
```

4. TECNOLOGÍAS UTILIZADAS

4.1. Stack Tecnológico Frontend

- **React 19:** Librería de interfaz gráfica declarativa de alto rendimiento.
- **TypeScript 5.x:** Superconjunto con tipado estático que reduce errores de tiempo de ejecución.
- **Vite:** Herramienta de compilación ultrarrápida (HMR) basada en ESM.
- **Tailwind CSS v3.4:** Framework CSS utilitario para diseño responsivo e institucional.
- **Leaflet.js:** Librería ligera para renderizado de mapas vectoriales e interactivos

del parque.

4.2. Stack Tecnológico Backend

- **Java 17 LTS:** Lenguaje principal de backend orientado a objetos y robustez.
- **Spring Boot 3.2:** Framework empresarial para APIs RESTful, inyección de dependencias y microservicios.
- **Spring Security:** Módulo de seguridad para autenticación con tokens JSON Web Tokens (JWT).
- **Spring Data JPA:** Persistencia relacional mediante Hibernate / Java Persistence API.

4.3. Herramientas de Integración y DevOps

- **Docker & Docker Compose:** Motor de contenedores para garantizar paridad entre entornos de desarrollo y producción.
- **Nginx:** Servidor web proxy inverso con terminación SSL/TLS y balanceo de carga.

5. ESTRUCTURA DEL PROYECTO

5.1. Estructura del Repositorio Backend (Spring Boot)

```
parque-barrondo-backend/
├── src/main/java/gt/gob/mcd/parquebarrondo/
│   ├── config/             # Configuración de Seguridad, JWT y CORS
│   ├── controller/         # REST Controllers (Endpoints API)
│   ├── dto/                # Data Transfer Objects (Requests/Responses)
│   ├── exception/          # Manejador de Excepciones Globales (@ControllerAdvice)
│   ├── model/              # Entidades JPA (tbl_usuarios, tbl_reservas, etc.)
│   ├── repository/         # Interfaces Spring Data JPA Repositories
│   └── service/            # Lógica de Negocio e Implementaciones
└── src/main/resources/
    ├── application.properties # Propiedades y Variables de Entorno
    └── db/migration/       # Scripts de Migración SQL (Flyway/Liquibase)
```

5.2. Estructura del Repositorio Frontend (React)

```
parque-barrondo-frontend/
├── src/
│   ├── assets/             # Imágenes institucionales, logotipos y mapas
```

```
│   ├── components/         # Componentes UI reutilizables (Botones, Tablas, Modales)
│   ├── context/            # React Context (AuthContext, ThemeContext)
│   ├── hooks/              # Custom Hooks (useAuth, useFetch)
│   ├── pages/              # Vistas principales (Inicio, Mapa, Reservas, Admin)
│   ├── services/           # Clientes HTTP (Axios API Client)
│   └── types/              # Definiciones TypeScript e Interfaces
├── index.html
└── vite.config.ts
```

6. CONFIGURACIÓN DEL ENTORNO

6.1. Requisitos Previos e Instalación con Winget

```
winget install --id Git.Git -e --source winget
winget install --id Microsoft.VisualStudioCode -e --source winget
winget install --id OpenJS.NodeJS.LTS -e --source winget
winget install --id Microsoft.OpenJDK.21 -e --source winget
winget install --id Microsoft.SQLServerManagementStudio.22 -e --
source winget
```

Nota: SQL Server Database Engine se descarga e instala por separado desde la página oficial de Microsoft seleccionando la versión SQL Server 2025 Standard Developer.

Para configurar el entorno de ejecución en un servidor de desarrollo o pruebas, asegúrese de declarar las siguientes propiedades en application.properties o mediante variables de entorno del sistema:

```
# Configuración de Base de Datos SQL Server
spring.datasource.url=jdbc:sqlserver://localhost:1433;databaseName=db_parque_barrondo;encrypt=
true;trustServerCertificate=true
spring.datasource.username=sa
spring.datasource.password=SecurePass2026!

# Configuración de JPA / Hibernate
spring.jpa.hibernate.ddl-auto=validate
spring.jpa.show-sql=false

# Seguridad JWT
jwt.secret.key=9a8b7c6d5e4f3a2b1c0d9e8f7a6b5c4d3e2f1a0b9c8d7e6f5a4b3c2d1e0f9a8b
jwt.expiration.ms=28800000
```

7. DESPLIEGUE DEL SISTEMA

El despliegue en producción se realiza mediante tecnología de contenedores Docker utilizando Docker Compose para la orquestación de servicios.

```
version: '3.8'
services:
  db:
    image: mcr.microsoft.com/mssql/server:2022-latest
```

```
    environment:
      - SA_PASSWORD=SecurePass2026!
      - ACCEPT_EULA=Y
    ports:
      - '1433:1433'

  backend:
    build: ./parque-barrondo-backend
    ports:
      - '8080:8080'
    depends_on:
      - db

  frontend:
    build: ./parque-barrondo-frontend
    ports:
      - '80:80'
```

8. MANTENIMIENTO PREVENTIVO

- **Estrategia de Backups:** Se deben programar respaldos automatizados de la base de datos SQL Server diariamente a las 01:00 AM (Diferenciales) y domingos (Completos).
- **Gestión de Logs:** Configurar rotación de logs mediante logrotate manteniendo un histórico máximo de 30 días para optimizar almacenamiento.

9. ANEXOS TÉCNICOS

9.1. Catálogo de Endpoints API REST

Método HTTP	Ruta Endpoint	Descripción / Propósito
POST	/api/v1/auth/register	Registro público de nuevos usuarios
POST	/api/v1/auth/login	Autenticación y generación de token JWT
GET	/api/v1/instalaciones	Consulta de catálogo y disponibilidad de áreas
POST	/api/v1/reservas	Creación de solicitud de reserva (Requiere Auth)
GET	/api/v1/admin/bitacora	Consulta de logs de auditoría (Solo Admin)